

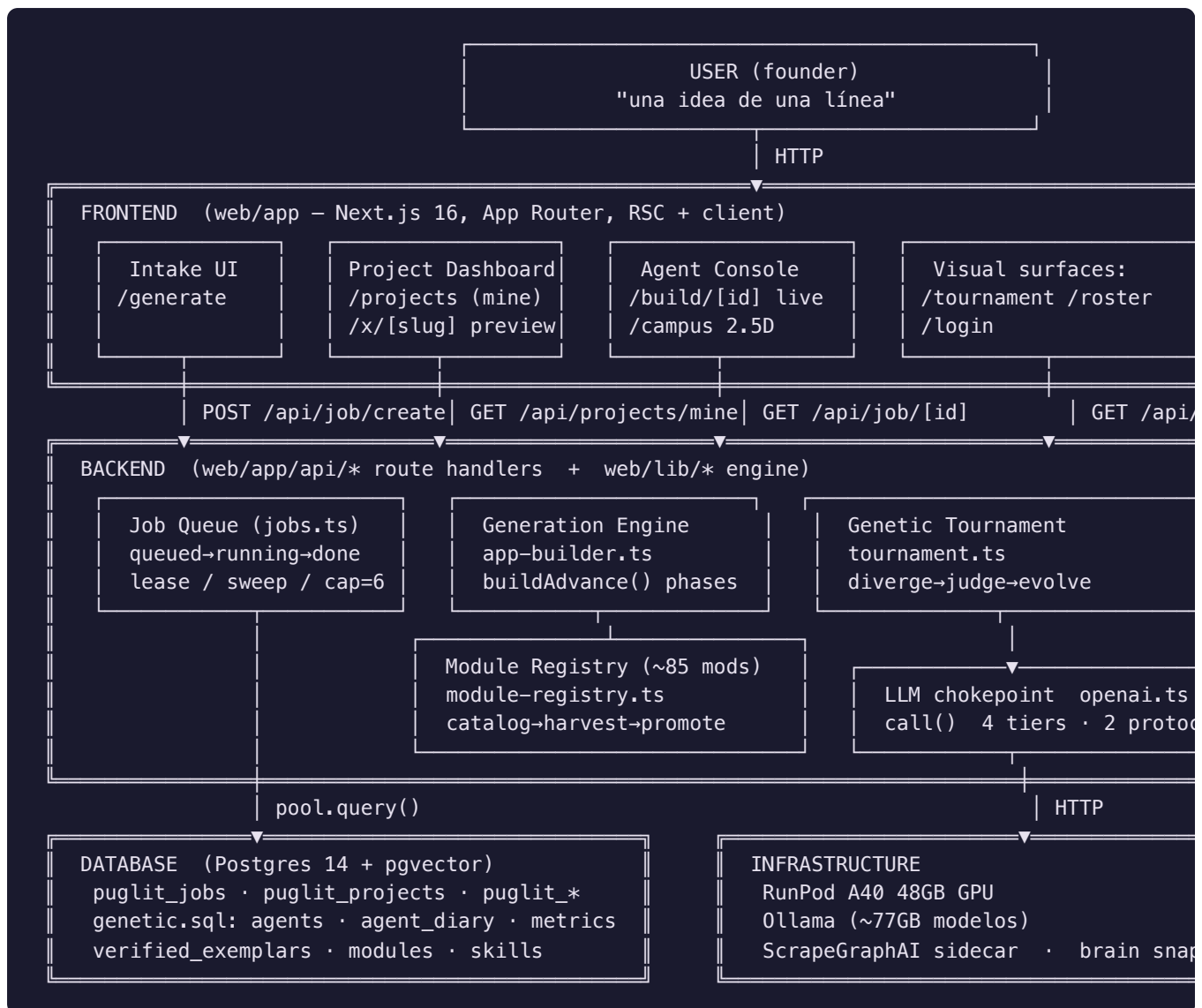
01 — System Architecture

Objetivo de este documento: explicar *absolutamente todo* el sistema Puglit de punta a punta — cómo una idea de una línea atraviesa el frontend, la cola de jobs, el torneo genético, el panel de jueces, la evolución, la finalización determinística, el build y la entrega — y cómo cada pieza (frontend, backend, base de datos, infraestructura) encaja y escala. Todo lo que sigue está verificado contra el código real en `/Users/alvaroz/projects/2026/puglit`. Cero placeholders.

Puglit es un sistema multi-agente **genético, self-hosted y agent-first** que genera apps SaaS full-stack completas (**Next.js 16 App Router + Turbopack + pg /Postgres crudo**) a partir de una idea de una línea. Corre **100% local sobre Ollama** (sin API key, sin costo) y soporta BYO de cualquier proveedor OpenAI-compatible o Anthropic. El chokepoint de toda inteligencia es una sola función, `call()` en `web/lib/openai.ts`.

1. High Level Overview

A nivel macro hay cuatro planos: el **Frontend** (la UI de intake, el dashboard de proyectos y la consola de agentes), el **Backend** (las API routes de Next bajo `web/app/api`, la *job queue* re-entrante de `web/lib/jobs.ts`, y el *module registry* viviente), la **Database** (Postgres + pgvector, con el cerebro genético en `web/sql/genetic.sql`), y la **Infraestructura** (RunPod A40 con Ollama + sidecars). Cada plano se comunica con el de abajo por contratos estrechos.



Una propiedad clave: **el backend nunca habla con un proveedor de LLM directamente**. Todo agente — el Domain Architect, los jueces, los Frontend/Backend engineers, los críticos— llama a `chatJSON` / `chatText`, que terminan en `call()` (`web/lib/openai.ts:197`). Eso permite mezclar proveedores por tier vía env (`PUGLIT_MODEL_*`, `PUGLIT_<TIER>_PROVIDER`) sin tocar el engine.

2. End-to-End Flow

El recorrido completo `User → Blueprint → Tournament → Judge → Evolution → Finalize → Build → Deploy`. El torneo (iteración 1) y la generación real (iteración 3) son fases distintas: el torneo elige el **diseño ganador** (un `Blueprint`), y la generación construye ese diseño con `initEngineStateWith(winnerBlueprint)` (`app-builder.ts:1286`, consumido en `jobs.ts:262`).

USER (POST /api/job/create → createJob → puglit_jobs status='queued')

ITERACIÓN 1 – DIVERGENCE (tournament.ts · divergeBlueprints)
3 equipos diseñan, cada uno con SU filosofía y SU modelo:
Team A "Lean" → qwen2.5-coder (TEAM_MODEL.A = MODELS.code)
Team B "Enterprise" → deepseek-coder-v2
Team C "Hacker" → devstral
cada uno → planBlueprint() → un Blueprint candidato

ITERACIÓN 1 – JUDGE (tournament.ts · judgeBlueprints)
Jurado (1 o varios modelos, "triumvirato") puntúa 4 áreas:
data · dev · design · business + crítica (rubric explícita 0-100)
overall = round(0.6 · judgeOverall + 0.4 · objectiveScore)
CIRCUIT BREAKER: si el jurado entero falla → degradación a draft
judge_agreement: métrica de acuerdo inter-juez (señal de ruido)
→ WINNER (TeamId)

EVOLUTION (progression.ts + skill-evolution.ts + genetic.sql)
XP = área × victoria × participación → niveles
diary: lessons/wins/critiques (quality 0-10, nomic-embed)
verified_exemplars: código que compiló+corrió, para retrieval futuro
SkillOpt: edits BOUNDED a los skill-docs, aceptados sólo si baten el
held-out validation set

▼ (winnerBlueprint inyectado al job)

ITERACIÓN 3 – BUILD (app-builder.ts · buildAdvance, re-entrante)
plan → critique → brief → routes → pages → FINALIZE → done
(plan se SALTEA si entró por initStateWith → arranca en critique)
FINALIZE: inyección determinística de ~85 módulos + quality gates

GOVERNANCE + QA (stakeholder.ts · 3 supervisiones × especialistas)
+ RUNTIME GATE local (build-local.mjs): boot real, smoke 0x5xx,
vitest+coverage, Queen evidence review

DELIVER / DEPLOY (jobs.ts · case "deliver" + ci-verify)
deliverToDisk → repo git local por usuario (generated/projects/<u>NNN</u>)
+ (si GITHUB_TOKEN) assembleProject → push → CI real → auto-repair
+ email "tu proyecto está listo" (Resend, opcional)

Cada paso del job es **avanzado por una llamada corta** a `advanceJob(id)` (cliente que poll-ea, o el cron `sweep`), porque el build completo es multi-minuto y no entra en un solo request serverless.

3. Core Components

3.1 Frontend

El frontend vive en `web/app` (Next.js 16, App Router). Tiene tres superficies principales más las visualizaciones del enjambre:

Superficie	Ruta	Rol
Intake UI	<code>app/generate</code>	Captura la idea (<code>name</code> , <code>what</code> , <code>audience</code> , <code>benefits</code> , <code>references</code> ...) y dispara <code>POST /api/job/create</code> .
Project Dashboard	<code>app/projects</code>	"Mis builds" del usuario, vía <code>GET /api/projects/mine</code> → <code>listUserJobs(email)</code> (<code>jobs.ts:162</code>).
Preview	<code>app/x/[slug]</code>	Vista del proyecto generado (<code>previewUrl = /x/<slug></code> , <code>jobs.ts:417</code>).
Agent Console	<code>app/build (/build/[id])</code>	Progreso <i>live</i> del job: pasos, estado, detalle; poll a <code>GET /api/job/[id]</code> .
Campus 2.5D	<code>app/campus</code>	Oficina pixel-art que anima el enjambre en vivo (<code>/api/genetic/tournament?live</code>).
Tournament / Roster	<code>app/tournament</code> , <code>app/roster</code>	Visualización del torneo y de los 75 agentes RPG.

El intake normaliza y acota cada campo antes de crear el job (`app/api/job/create/route.ts` : `name` a 60 chars, `what` a 240, `references` a 8000, etc.) y exige sesión (`getSession()`) salvo *service request* interno.

3.2 Backend

El backend son las **API routes** bajo `web/app/api` (28 handlers) y el **engine** en `web/lib` .

```
app/api
├─ job/create          POST → createJob()          (encola un build)
├─ job/[id]           GET  → getJob() / advanceJob() (poll + avance del job)
├─ generate · spec · interview · references · designs      (intake / spec)
├─ build              POST → arranque de build
├─ genetic/
│   └─ tournament     POST/GET → runDivergence + status (iteración 1)
│   └─ build · export · seed · modules · agents · logo    (genetic ops)
├─ admin/brain · admin/evolve-skills                      (brain ops, Skill0pt)
├─ cron/sweep        GET  → sweep()                   (watchdog/headless)
├─ auth/{request,verify,me} · demo/{auth,me,records}      (spine auth)
└─ doctor · examples · waitlist · projects/mine
```

Job Queue (`web/lib/jobs.ts`). Una cola con barandas:

Constante	Valor	Significado
CONCURRENCY	6	máximo de jobs corriendo a la vez (el resto queda <code>queued</code>)
STEP_TIMEOUT_MS	120 000	un step <code>running</code> más viejo que esto se considera trabado
MAX_ATTEMPTS	3	reintentos por step antes de marcarlo <code>error</code>
LEASE_MS	100 000	ventana de lock mientras se avanza un step
LIVE_MS	300 000	un job <code>running</code> sólo ocupa slot si se actualizó hace poco
CI_STEP_TIMEOUT_MS	900 000	ci-verify/engine/stakeholder corren minutos: lease más larga

El pipeline declarado en `PLAN` son **24 steps** (`data-model`, `research`, `contracts`, `brand`, `design`, `schema`, `seed`, `erd`, `auth`, `payments`, `email`, `crons`, `admin`, `dashboard`, `analytics`, `seo`, `security`, `engine`, `stakeholder`, `docs-tech`, `docs-biz`, `env`, `deliver`, `ci-verify`). Cada step es honesto sobre su naturaleza: *GENERATED* per producto, *ASSEMBLED* del spine, o *SCAFFOLD*.

Tres steps son **re-entrantes** (`advanceJob` los maneja antes del `switch`): `engine` (una unidad de `buildAdvance` por llamada), `stakeholder` (una wave de supervisión por llamada) y `ci-verify` (un poll/fix de CI por llamada). El resto del trabajo concurrente lo empuja `sweep()` (`jobs.ts:438`): promueve la cola, recupera steps trabados (lease vencido + demasiado viejo) y avanza cada job vivo en paralelo —pensado para un cron por minuto (`/api/cron/sweep`)— así el build avanza **aunque nadie tenga abierta la página**.

El lock es un **lease en Postgres** (`acquireLease`, `jobs.ts:196`): un `UPDATE ... SET lease_until = NOW() + interval ... WHERE lease_until IS NULL OR lease_until < NOW() RETURNING id`. El poller y el sweeper nunca avanzan el mismo job simultáneamente.

Generation Engine (`web/lib/app-builder.ts`). La máquina de estados `buildAdvance` (ver §4) que deriva el `Blueprint` (tablas + rutas + páginas) y emite cada archivo con un agente especializado.

Module Registry (`web/lib/module-registry.ts`). ~85 módulos builtin de inyección **determinística**: cada `deterministicX(config, bp) → {files, extraSql}` se inyecta en `finalize` cuando matchean keywords/capabilities. Registry **viviente**: `catalog` (los agentes lo VEN) → `harvest` (los agentes lo EXTIENDEN con código que escribieron) → `promote` (experimental → stable → core). Verticales parametrizadas al nombre de tabla detectado (cero hardcoding), p.ej. `rentals-module.ts` (anti-double-booking con `EXCLUDE USING gist`).

LLM chokepoint (`web/lib/openai.ts`). Cuatro tiers de modelo —`premium` (arquitectura, juicio, review), `balanced`, `cheap` (extracción), `code` (rutas/páginas/repair)— y **dos protocolos**: `openai` (OpenAI/Gemini/Ollama/custom, misma forma) y `anthropic` (adaptador a la Messages API). El proveedor por defecto se resuelve solo: sin nada configurado, corre 100% local en Ollama (`gemma2:27b` / `qwen2.5-coder:7b`). Cada tier puede apuntar a un proveedor distinto vía env.

```
agente → chatJSON()/chatText() → call()
(web/lib/openai.ts)
└─ protocol "openai" → /chat/completions
    (OpenAI · Gemini · Ollama · custom)
    · json_schema strict si hay schema
    · Ollama nativo /api/chat con `format`
└─ protocol "anthropic" → /messages (adaptador)

cada call → traceCall() (tier, prompt size, latency, ok) → run-profile (agent-house)
```

3.3 Database

Postgres 14 + **pgvector**, accedido vía `pg` crudo. `web/lib/db.ts` crea un `Pool` lazy hacia el pooler de Supabase (Supavisor, IPv4; TLS salvo `POSTGRES_SSL=disable`). `query(text, params)` es el helper compartido; **todo es parametrizado** (`$1, $2, ...`).

Tablas operativas (de `db.ts`, `SCHEMA_SQL`):

Tabla	Para qué
<code>puglit_jobs</code>	un job = el pipeline completo: <code>status</code> , <code>steps</code> (JSONB), <code>artifacts</code> (JSONB), <code>lease_until</code> , <code>user_email</code> , <code>completion</code>
<code>puglit_projects</code>	proyectos generados (<code>slug</code> , <code>config</code> JSONB, <code>landing_html</code>)
<code>puglit_waitlist</code>	waitlist

El **cerebro genético** vive en `web/sql/genetic.sql`: `puglit_agents`, `puglit_agent_diary` (lessons/wins con embeddings nomic-embed), `puglit_metrics`, `verified_exemplars`, `puglit_modules`, `puglit_skills`, `puglit_skill_rejects`. El diario y los exemplars usan pgvector para retrieval por similitud (relevance floor 0.35 + recency decay + quality $\geq$ 45 + outcome $\neq$ failure, anti-poisoning).

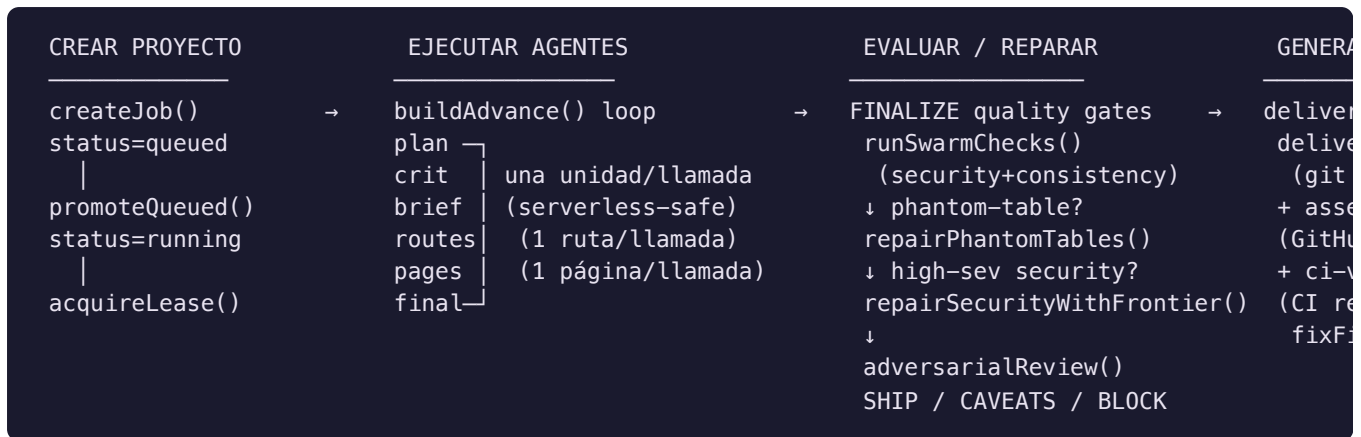
3.4 Infrastructure

Componente	Detalle
GPU box	RunPod A40 48GB ; Ollama con ~77GB de modelos open-weight (gemma2, qwen2.5-coder, deepseek-coder-v2, devstral, nomic-embed)
DB	Postgres 14 + pgvector
Sidecar de scraping	<code>infra/scrapegraph-sidecar.py</code> (ScrapeGraphAI sobre Ollama local) — usado por <code>groundReferences</code> para extraer estructura real de productos de referencia
Gateways	el engine asume gateways/thin-clients para capacidades externas (scraper, conectores omnicanal); el spine no trae object storage ni websockets por diseño
Brain persistence	<code>web/lib/brain-sync.ts</code> : tablas append-only por UNION (CRDT grow-set), skills mutables arbitradas por <code>val_score</code> ; cloud Postgres autoritativo + snapshots a git/bucket (<code>infra/brain-snapshot.sh</code>)

El spine (`spine/`) se ensambla en cada app generada y aporta la base determinística: auth, rate-limit, analytics, i18n, mailer y **db** (`export const pool = new Pool()`). Las constraints duras del generador derivan de las capacidades reales del spine: sin deps npm externas, realtime = polling, imágenes = data-URLs en `TEXT`, SQL parametrizado por `pool.query`.

4. Runtime Lifecycle

El ciclo de vida real de un build, fase por fase. La máquina de estados es `EngineState` (`app-builder.ts:1270`) con fases `plan` → `critique` → `brief` → `routes` → `pages` → `finalize` → `done`, avanzada una unidad por llamada.



Crear proyecto. `createJob` (`jobs.ts:145`) inserta el job en `puglit_jobs` con `status='queued'` y los 24 steps en `pending`. `promoteQueued` lo promueve a `running` bajo el cap de 6 (contando sólo jobs vivos: un job huérfano envejece y libera su slot).

Ejecutar agentes. El step `engine` corre `buildAdvance` una **unidad bounded** por llamada: - `plan` → `planBlueprint()`: el Domain Architect infiere `kind` (`public` | `accounts`), tablas, rutas y páginas del producto real (no un CRUD genérico). Si entró por torneo, se saltea. - `critique` → `critiqueBlueprint()` agrega rutas/páginas faltantes, garantiza creación de contenido y homepage, y deduplica. - `brief` → `genDesignBrief()`: el Art Director emite una identidad visual bespoke; se materializa `sql/app.sql` y `docs/DESIGN.md`. - `routes` → **una ruta por llamada** (`genRouteFile`): un Backend Engineer escribe un `route.ts` con todos sus métodos; `routeScore()` puntúa objetivamente (handlers, SQL parametrizado, sin interpolación) para best-of-N. Cada ruta pasa por `hardenRoute`. - `pages` → **una página por llamada** (`genPage`): un Frontend Engineer escribe la página siguiendo el brief, con loop reactivo de QC visual (`reviewPageVisual` → `refinePage`, máx 2 ciclos).

Evaluar. En `finalize` (`app-builder.ts:1342`) se inyectan los ~85 módulos determinísticos, se hace `harvestModules` (el directorio crece con el trabajo del enjambre), se generan tests vitest (`genTests`), y corren los gates: - `runSwarmChecks(files, declaredTables)` — escaneo estático de seguridad + consistencia (secrets / SQLi / phantom-table). - `adversarialReview` — patrón adverse: 3 lentes ortogonales (Auditor / Adversary / Pragmatist) → cross-examination (≥2 lentes = confirmado) → veredicto SHIP / SHIP-WITH-CAVEATS / BLOCK.

Reparar. Si hay phantom-tables → `repairPhantomTables` (infiere el schema de la intención) y re-escanea. Si hay seguridad high-severity → `repairSecurityWithFrontier` (presupuesto bounded de un modelo más fuerte). El `adversarialReview` también hace bounded-repair de los CRITICAL confirmados. Cuando el gate estático queda limpio (`highIssues === 0`), las rutas/páginas se guardan como `verified_exemplars` para subir el piso de builds futuros sin un modelo mejor.

Después, el step `stakeholder` corre la **governance** (`stakeholder.ts`, `initStakeholderState(3)`): 3 supervisiones × especialistas + los fixers de la Queen, una wave por llamada. Y el **RUNTIME GATE local** (`web/scripts/build-local.mjs`): ensambla spine + archivos bespoke, carga el SQL en una DB fresca (reparando DDL), corre `tsc` en loop hasta verde, `vitest --coverage`, la **Queen evidence review** (mide coverage real, rebota bajo el bar `COV_BAR`), y el `smokeTest()` que **bootea la app real y exige 0x5xx**. Es el chequeo "¿realmente corre?" que sólo un compile verde no da: en el litmus Stayforge expuso 7 bugs reales invisibles a un compile verde.

Generar / entregar. El step `deliver` (`jobs.ts:392`) persiste el proyecto, hace `deliverToDisk` (repo git local por usuario en `generated/projects/<user>/<NNN-slug>/`, **sin compilar**) y, si hay `GITHUB_TOKEN`, `assembleProject` (spine + bespoke + harness + docs) y push. El step `ci-verify` re-entrante dispara CI real, poll-ea, y hasta `MAX_CI=3` ciclos de `fixFiles` (auto-reparación de errores `tsc`) antes de entregar con reporte. Email opcional por Resend.

5. Data Flow

5.1 Inputs

```
IntakeAnswers (app/api/job/create)
  name · what · audience · benefits[] · color · languages · monetization
  price · modules[] · references? · archetype?
    |
    |→ generateConfig() + applyBranding() → DomainConfig (identity, palette, entities)
    |→ designEntities()                  → entidades inferidas
    |→ runContracts()                    → tipos + contrato de API
    |→ researchProduct() / studyReference() → plan de ingesta + producto de referencia
    |→ groundReferences() (ScrapeGraphAI) → estructura real del producto clonado
```

5.2 Outputs

```
buildAdvance → state.files: AppFile[] (path + content)
  sql/app.sql (schema bespoke, corre tras 001/002/003 del spine)
  app/api/**/route.ts (handlers reales, parametrizados)
  app/**/page.tsx (páginas reales, premium, defensivas)
  app/app/layout.tsx (shell bespoke con auth-gate, sólo kind=accounts)
  lib/__tests__/domain.test.ts (tests vitest de la lógica pura)
  + ~85 módulos determinísticos según capabilities
  docs/ (ER diagram Mermaid, contracts, BUILD_REPORT, technical + business docs)
  harnessFiles() (docker, db-setup, scripts/e2e.mjs, RUN-AND-TEST.md)
```

5.3 Persistencia

Todo el estado del build vive en `puglit_jobs.artifacts` (JSONB): `engineState`, `blueprint`, `appFiles`, `stakeholderState`, `contracts`, `research`, `reference`, `ci`. Eso es lo que hace que el pipeline sea **resumible** en serverless: cada `advanceJob` lee el estado persistido, avanza una unidad y lo re-escribe con `persist()` (que recalcula `completion = % steps done` y libera el lease).

El **conocimiento genético** se persiste aparte, en las tablas de `genetic.sql`, y sobrevive entre builds: diary, exemplars, metrics, skills. `brain-sync.ts` lo trata como un grow-set CRDT (append-only por UNION) con snapshots a git/bucket — el cloud Postgres es la fuente autoritativa.


```

advanceJob() → [ puglit_jobs → ]
                  steps (JSONB)  artifacts (JSONB: engineState, blueprint, files...)
                  status · lease_until · completion · user_email
                  (resumible: leer → avanzar 1 unidad → re-escribir)

genetic.sql (cerebro, cross-build): agents · agent_diary(pgvector) · metrics ·
verified_exemplars(pgvector) · modules · skills · skill
└ brain-sync.ts (CRDT grow-set) → snapshots git/bucket

```

6. Scalability Model

El modelo de escala se apoya en tres palancas reales del código: el **cap de concurrencia** (`CONCURRENCY = 6`), el **lease en Postgres** (cualquier worker puede tomar cualquier job sin pisarse), y el **avance por unidad bounded** (cada llamada cabe en un budget serverless). Como cada job sólo ocupa slot si está *vivo* (`LIVE_MS`), la cola se autolimpia.

throughput ▲

worker + GPU

(limitado por la GPU: Ollama corre 1 modelo a la vez, los jueces y equipos se ejecutan SECUENCIALMENTE)

horizontal: + workers golpeando /api/cron/sweep

cada uno toma jobs por LEASE; CONCURRENCY acota in-flight

1 100 1 000 10 000 → usuarios

Escala	Cuello de botella dominante	Cómo lo absorbe la arquitectura actual
Single user	Latencia del modelo local en 1 GPU	Un solo job ocupa la GPU; el avance por unidad mantiene cada request corto. Local-first: sin costo de API. El RUNTIME GATE corre en la misma caja.
100 users	Profundidad de cola con <code>CONCURRENCY=6</code>	Los jobs encolan (<code>queued</code>) y <code>promoteQueued</code> los va promoviendo bajo el cap; <code>sweep</code> (cron 1/min) los empuja headless. Latencia sube, nada se pierde. La GPU es serial → throughput ≈ 6 builds concurrentes.
1 000 users	GPU única (Ollama serial) + ancho de Postgres	Escalar horizontal el cómputo de modelos : apuntar tiers a proveedores hosteados o a un pool de boxes Ollama vía <code>PUGLIT_<TIER>_PROVIDER / _BASE_URL</code> (el chokepoint <code>call()</code> lo hace transparente). El lease permite N workers sin coordinación extra. Subir <code>CONCURRENCY</code> y <code>POSTGRES_POOL_MAX</code> por worker.
10 000 users	Coordinación de cola + persistencia del cerebro	Cola distribuida sobre el mismo lease (varios sweepers); Postgres con pooler (Supervisor) y réplicas de lectura para dashboards. El cerebro genético es CRDT grow-set (<code>brain-sync.ts</code>): mergea entre nodos por UNION, sin lock global; snapshots a bucket. Tiers premium/code a frontier providers; cheap a un fleet local.

Notas de diseño que habilitan esto, todas verificadas: - *Stateless por request*. `advanceJob` no guarda estado en memoria del proceso (salvo el mapa in-memory del torneo, que es por-proceso y reconstruible). Todo lo durable está en Postgres. - *Idempotencia*. `saveProject` y `deliverToDisk` son re-ejecutables

(upsert `ON CONFLICT`); un step puede reintentar `MAX_ATTEMPTS` veces sin corromper. - *Degradación graceful*. Si falta la DB, el generador sigue devolviendo config (no persiste, lo dice). Si falla el jurado entero, el circuit breaker degrada a draft en vez de bloquear. Si falta GitHub, entrega a disco igual. - *Costo cero por defecto*. Toda la inteligencia es local (Ollama); escalar el costo es una decisión explícita de apuntar tiers a un proveedor pago, no un requisito.

Apéndice — Mapa de archivos citados

Archivo	Rol en la arquitectura
<code>web/lib/app-builder.ts</code>	Engine de generación: <code>EngineState</code> , <code>buildAdvance</code> , fases, <code>planBlueprint</code> , finalize
<code>web/lib/jobs.ts</code>	Job queue: <code>PLAN</code> (24 steps), <code>createJob</code> , <code>advanceJob</code> , <code>sweep</code> , leases, re-entrancia
<code>web/lib/openai.ts</code>	Chokepoint LLM: <code>call()</code> , <code>MODELS</code> (4 tiers), 2 protocolos, resolución de proveedor
<code>web/lib/db.ts</code>	Pool Postgres, <code>query</code> , <code>SCHEMA_SQL</code> (<code>puglit_jobs</code> / <code>puglit_projects</code> / <code>puglit_waitlist</code>)
<code>web/lib/tournament.ts</code>	Torneo genético: <code>divergeBlueprints</code> , <code>judgeBlueprints</code> (0.6/0.4), circuit breaker, agreement
<code>web/lib/module-registry.ts</code>	Registry viviente de ~85 módulos: catalog → harvest → promote
<code>web/scripts/build-local.mjs</code>	RUNTIME GATE local: boot + smoke 0x5xx, tsc loop, vitest+coverage, Queen review
<code>web/sql/genetic.sql</code>	Cerebro genético: agents, diary, metrics, exemplars, modules, skills
<code>web/app/api/**</code>	28 route handlers (job, genetic, auth, cron/sweep, admin/brain...)